

Actividad 3 - GRUPO 3

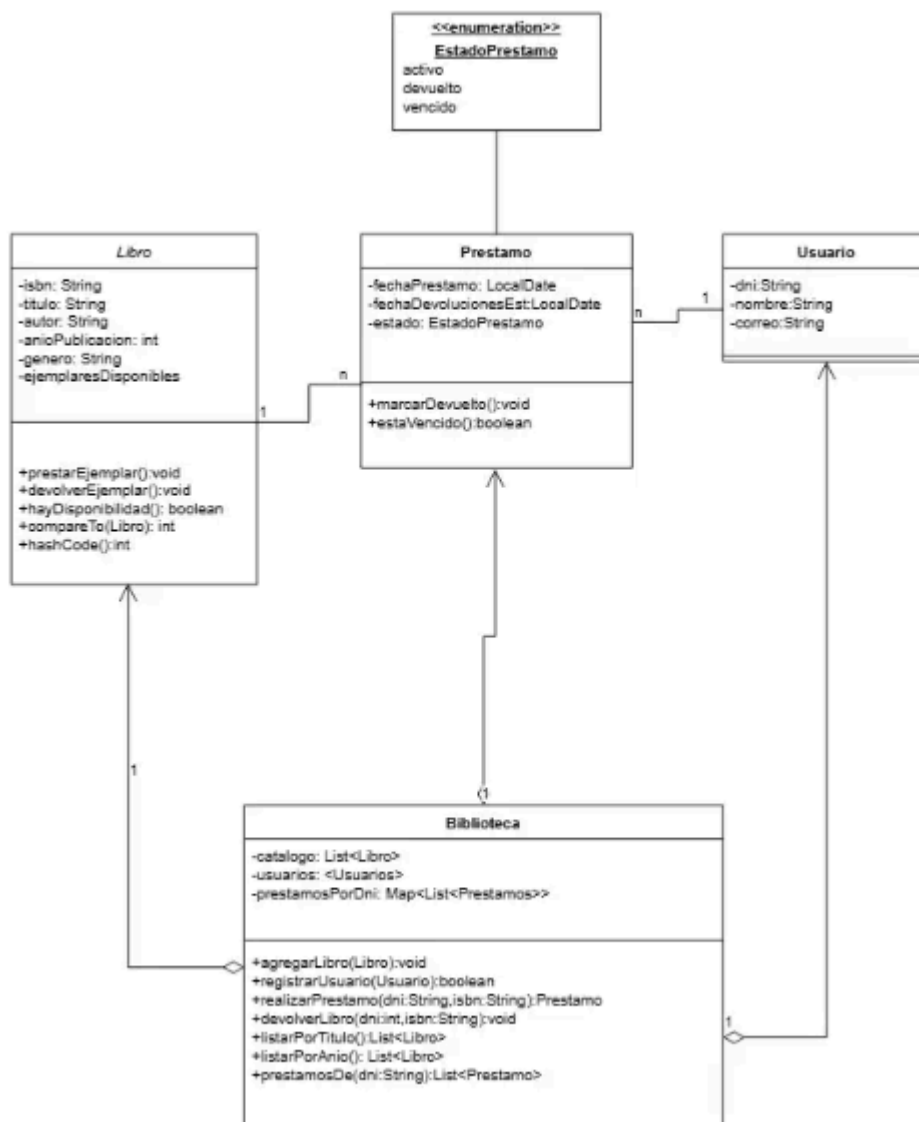
Paradigma orientado a objetos

Integrantes:

- Thiago Vazquez Grethe LU: 1180354
- Juan Ignacio Suarez LU:1150935
- Daza Ivan LU: 1146519
- Rebollo Pedro LU: 1166436
- Maria Victoria Molina LU: 1163150
- Tobias Fernandez Sarso LU: 1153411
- Michailenko Ivan LU: 1142765
- Laretta Micaela LU: 1067846

Actividad 3 — Sistema de Gestión de Biblioteca I

1. Diagrama de clases UML:



2. Uso de colecciones - Justificación

List<Libro>:

La colección catalogo utiliza una List porque permite almacenar todos los libros de la biblioteca manteniendo un orden y permitiendo recorridos sencillos.

Set<Usuario>:

La colección usuarios utiliza un Set porque evita registrar usuarios duplicados. El criterio de igualdad elegido es el DNI, ya que dos usuarios con el mismo DNI representan a la misma persona.

Map<String,List<Prestamo>>

La colección prestamosPorDni utiliza un Map donde:

La clave es el DNI del usuario y el valor es la lista de préstamos realizados por ese usuario. Esto permite acceder rápidamente a los préstamos de una persona sin recorrer toda la colección.

Los arreglos no fueron utilizados porque tienen tamaño fijo y resultan menos flexibles cuando la cantidad de libros, usuarios y préstamos cambia constantemente.

3. Comparable y Comparator

Comparable: La clase Libro implementa Comparable<Libro> mediante el método compareTo(). El orden natural elegido es:

1. Título.
2. Autor en caso de empate.

Ejemplo:

```
```java
@Override
public int compareTo(Libro otro){

 int resultado =
 this.titulo.compareToIgnoreCase(otro.titulo);

 if(resultado == 0){
 resultado =
 this.autor.compareToIgnoreCase(otro.autor);
 }

 return resultado;
}
```
```

Comparator

Se implementó la clase ComparadorPorAnio para ordenar los libros según el año de publicación. Se eligió un orden descendente, mostrando primero los libros más recientes.

```
```java
public class ComparadorPorAnio
 implements Comparator<Libro>{

 @Override
 public int compare(Libro l1, Libro l2){
```

```

 return Integer.compare(
 l2.getAnioPublicacion(),
 l1.getAnioPublicacion());
 }
}
...

```

Comparable define el orden natural propio de la clase Libro mientras que Comparator permite crear criterios alternativos de ordenamiento sin modificar la clase.

#### 4. Recorridos con cursores

Para recorrer los préstamos de un usuario y eliminar aquellos que ya fueron devueltos se utiliza Iterator.

```

...java
Iterator<Prestamo> it =
 prestamos.iterator();

while(it.hasNext()){

 Prestamo p = it.next();

 if(p.getEstado() ==
 EstadoPrestamo.DEVUELTO){

 it.remove();
 }
}
...

```

Se utiliza Iterator porque permite eliminar elementos de forma segura durante el recorrido. Si se eliminan elementos dentro de un for-each, Java puede lanzar una ConcurrentModificationException porque la colección se modifica mientras está siendo recorrida.

#### 5. Manejo de excepciones y validacion de datos

DatosInvalidosException, Se utiliza cuando:

- El ISBN está vacío
- El título está vacío.
- El año de publicación es inválido.
- La cantidad de ejemplares es negativa.

**LibroNoDisponibleException:** Se utiliza cuando se intenta prestar un libro que no posee ejemplares disponibles.

**UsuarioNoRegistradoException:** Se produce cuando se intenta realizar una operación con un usuario inexistente.

**LibroInexistenteException:** Se produce cuando se intenta buscar un libro que no existe en el catálogo.

Ejemplo de try-catch-finally:

```

```java
try {

    biblioteca.realizarPrestamo(
        "30111222",
        "9789871234567");

}
catch(LibroNoDisponibleException e){

    System.out.println(
        "No hay ejemplares disponibles");

}
catch(UsuarioNoRegistradoException e){

    System.out.println(
        "Usuario inexistente");

}
finally{

    System.out.println(
        "Intento de préstamo finalizado");

}
```

```

El bloque finally se ejecuta siempre, independientemente de si ocurrió una excepción o no.

## 6. Prueba de integración del programa

### Creación de libros

```
```java
Libro l1 = new Libro("111","El Principito",
    "Saint Exupery",1943,"Novela",2);

Libro l2 = new Libro("222","1984",
    "George Orwell",1949,"Novela",1);

Libro l3 = new Libro("333","Rayuela",
    "Julio Cortazar",1963,"Novela",3);

Libro l4 = new Libro("444","Martin Fierro",
    "Jose Hernandez",1872,"Poesia",1);
...
```
```

### Creación de usuarios

```
```java
Usuario u1 =
    new Usuario("30111222",
        "Juan Perez",
        "juan@gmail.com");

Usuario u2 =
    new Usuario("30222333",
        "Ana Lopez",
        "ana@gmail.com");

Usuario u3 =
    new Usuario("30333444",
        "Pedro Diaz",
        "pedro@gmail.com");
...
```
```

### Registro de usuarios

```
```java
biblioteca.registrarUsuario(u1);
biblioteca.registrarUsuario(u2);
biblioteca.registrarUsuario(u3);
```
```

```
biblioteca.registrarUsuario(u1);
...
```

El segundo registro de u1 no se realiza porque el Set evita usuarios duplicados.

### Préstamo válido

```
```java
biblioteca.realizarPrestamo(
    "30111222",
    "111");
...

```

La cantidad de ejemplares disponibles disminuye en una unidad.

Préstamo inválido

```
```java
try{

 biblioteca.realizarPrestamo(
 "30222333",
 "222");

 biblioteca.realizarPrestamo(
 "30333444",
 "222");

}
catch(LibroNoDisponibleException e){

 System.out.println(e.getMessage());
}
...

```

Se genera una excepción al intentar prestar un libro sin stock.

### Listado ordenado

```
```java
biblioteca.listarPorTitulo();
biblioteca.listarPorAnio();
...

```

Recorrido de préstamos

```
``java
for(Prestamo p :
    biblioteca.prestamosDe("30111222")){

    System.out.println(p);
}
``
```